Screenshot%202021-02-04%20at%2010.30.29%20am.png

Convolutional Neural Networks (CNN) - Object Recognition

Imports

```
from numpy.random import seed
seed(888)

#from tensorflow import set_random_seed
#set_random_seed(4112)
import tensorflow
tensorflow.random.set_seed(112)
```

```
import os
import numpy as np
import itertools

import tensorflow as tf
import keras
from keras.datasets import cifar10 # importing the dataset

from keras.models import Sequential       #to define model/ layers
from keras.layers import Dense, Conv2D, MaxPool2D, Flatten

from sklearn.metrics import confusion_matrix

# To Explore the images
from IPython.display import display
from keras.preprocessing.image import array_to_img

from tensorflow.keras.utils import to_categorical

import matplotlib.pyplot as plt
%matplotlib inline
```

```
import pandas as pd
```

We are using Tensorflow to power Keras

Get the Dataset

CIFAR-10 is an established computer-vision dataset used for object recognition. It is a subset of the 80 million tiny images dataset and consists of 60,000 32x32 color images containing one of 10 object classes, with 6000 images per class. It was collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. The dataset is popularly used to train image classification models

Screenshot%202021-02-05%20at%2012.47.25%20pm.png

```
# Getting the dataset as a Tuple

(x_train_all, y_train_all), (x_test, y_test) = cifar10.load_data()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-pyth
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 5s 0us/step
```

Constants

```
LABEL_NAMES = ['airplane', 'automobile','bird','cat', 'deer', 'dog'
```

Exploring the Data

Lets look at the first image in the dataset

```
x_train_all.shape
```

```
(50000, 32, 32, 3)
```

```
x_train_all[0]
```

```
ndarray (32, 32, 3) show data
```

```
x_train_all[0].shape
```

```
(32, 32, 3)
```

Using ipython to display the image

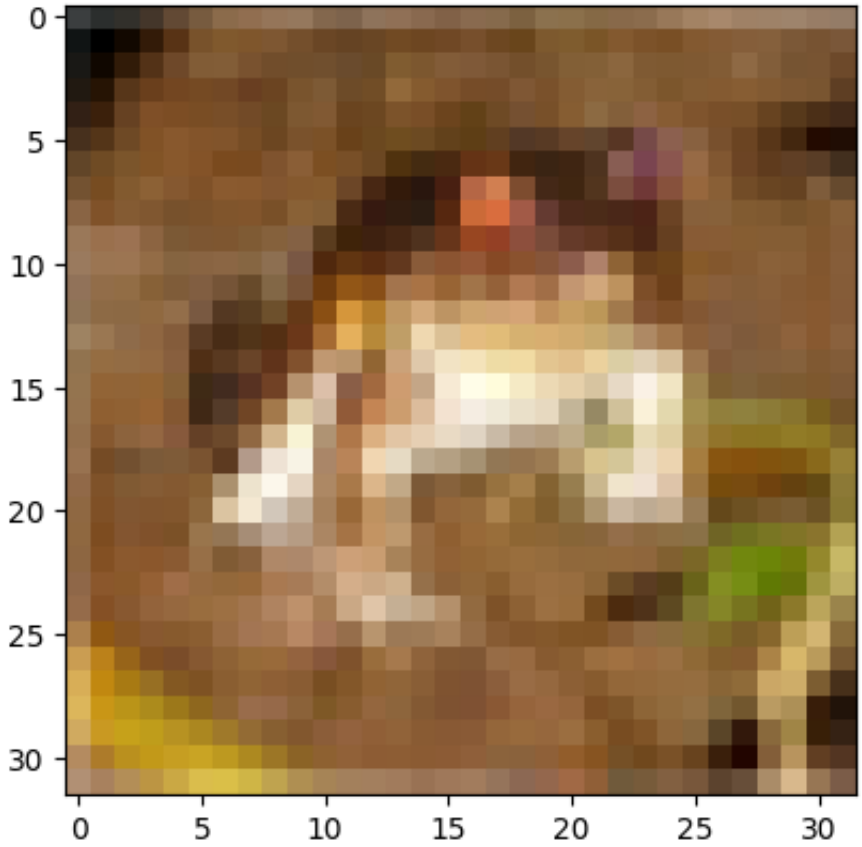
```
# To use the ipython display to view an image

pic = array_to_img(x_train_all[0])
display(pic)
```

Using Matplotlib to view the image

```
plt.imshow(x_train_all[0])
```

```
<matplotlib.image.AxesImage at 0x7b0e1f57c950>
```




```
# To check the label
y_train_all.shape
```

```
(50000, 1)
```

```
# Note that in the image above the index 1 corresponds to "Automobi
# we have a 2 dimension numpy array; that is why we also include "

y_train_all[0][0]
```

```
np.uint8(6)
```

```
# Using the lable names to get the actual names of classes

LABEL_NAMES[y_train_all[0][0]]
```

```
'frog'
```

The shape of the image

```
* 32, 32 is the weight and the height
* 3 is the number of channels (These are the number of colors): Red, G
```

- x_train_all.shape >>> (50000, 32, 32, 3)
  - this means we have 50,000 entries | then 32x32 weight and height| 3 colors (RGB)

```
x_train_all.shape
```

```
(50000, 32, 32, 3)
```

```
number_of_images, x, y, c = x_train_all.shape
print(f'Number of images = {number_of_images} \t| width = {x} \t| h
```

```
Number of images = 50000        | width = 32    | height = 32   | ch
```

```
x_test.shape
```

```
(10000, 32, 32, 3)
```

Preprocess Data

* We need to preprocess our data so that it is easier to feed it to our neural network.

Scalling both x_train and test

```
x_train_all =x_train_all / 255.0
```

```
x_test =  x_test / 255.0
```

```
y_test
```

```
array([[3],
       [8],
       [8],
       ...,
       [5],
       [1],
       [7]], dtype=uint8)
```

Creating categorical encoding for the "y " data

```
# 10 >>> simply means we have 10 classes like we already know (crea
y_cat_train_all = to_categorical(y_train_all,10)
```

```
# 10 >>> simply means we have 10 classes like we already know (crea
y_cat_test = to_categorical(y_test,10)
```

```
y_cat_train_all
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 0., ..., 0., 0., 1.],
       ...,
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 1., 0., ..., 0., 0., 0.],
       [0., 1., 0., ..., 0., 0., 0.]])
```

Creating the Validation dataset

Screenshot%202021-02-05%20at%206.37.47%20pm.png

For small data we usually go with: * 60% for Training * 20% Validation * 20% Testing

Only the final selected model gets to see the testing data. This helps us to ensure that we have close to real data in real-world when the model is deployed. Only our best model gets to see our testing dataset. Because it will give us a realistic impression of how our model will do in the real world

---

However, if the dataset is enormous.: * 1% for is used for validation * 1% for is used for testing

```
VALIDATION_SIZE = 10000
```

```
# VALIDATION_SIZE = 10,000 as defined above

x_val = x_train_all[:VALIDATION_SIZE]
y_val_cat = y_cat_train_all[:VALIDATION_SIZE]
x_val.shape
```

```
(10000, 32, 32, 3)
```

```
y_val_cat
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 0., ..., 0., 0., 1.],
       ...,
       [0., 1., 0., ..., 0., 0., 0.],
       [0., 1., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

NEXT:

- We Create two NumPy arrays x_train and y_train that have the shape(40000, 3072) and (40000,1) respectively.
- They will contain the last 40000 values from x_train_all and y_train_all respectively

```
x_train = x_train_all[VALIDATION_SIZE:]
y_cat_train= y_cat_train_all[VALIDATION_SIZE:]
```

```
x_train.shape
```

```
(40000, 32, 32, 3)
```

```
y_cat_train
```

```
array([[0., 1., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 1., 0., ..., 0., 0., 0.],
       [0., 1., 0., ..., 0., 0., 0.]])
```

Screenshot%202021-02-15%20at%208.04.47%20pm.png

NOTE:

* *FILTERS:* Typical values for the number of filters can be determined by the data set's complexity. So essentially the larger the images, the more variety and the more classes you're trying to classify then the more filters you should have.

* Most times people typically pick filter based on powers of 2, for example, 32. However, if you have more complex data like road signs etc. you should be starting with a higher filter value

The default STRIDE value is 1 x 1 pixel

BUILDING THE MODEL

```
model = Sequential()

## ************* FIRST SET OF LAYERS ************************

# CONVOLUTIONAL LAYER
model.add(Conv2D(filters=32, kernel_size=(4,4),input_shape=(32, 32,
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

## *************** SECOND SET OF LAYERS **********************
#Since the shape of the data is 32 x 32 x 3 =3072 ...
#We need to deal with this more complex structure by adding yet ano

# *************CONVOLUTIONAL LAYER
model.add(Conv2D(filters=32, kernel_size=(4,4),input_shape=(32, 32,
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

# FLATTEN IMAGES FROM 32 x 32 x 3 =3072 BEFORE FINAL LAYER
model.add(Flatten())

# 256 NEURONS IN DENSE HIDDEN LAYER (YOU CAN CHANGE THIS NUMBER OF
model.add(Dense(256, activation='relu'))

# LAST LAYER IS THE CLASSIFIER, THUS 10 POSSIBLE CLASSES
model.add(Dense(10, activation='softmax'))


model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolution
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	P
conv2d (Conv2D)	(None, 29, 29, 32)	
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	
conv2d_1 (Conv2D)	(None, 11, 11, 32)	
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 32)	
flatten (Flatten)	(None, 800)	
dense (Dense)	(None, 256)	2
dense_1 (Dense)	(None, 10)	

Total params: 225,610 (881.29 KB)
Trainable params: 225,610 (881.29 KB)
Non-trainable params: 0 (0.00 B)

Adding Early stopping

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stop = EarlyStopping(monitor='val_loss',patience=2)
```

```
history = model.fit(x_train,y_cat_train,epochs=25,validation_data=(
```

```
Epoch 1/25
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 61s 46ms/step - accuracy: 0.4324 - lo
Epoch 2/25
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 54s 43ms/step - accuracy: 0.5522 - lo
Epoch 3/25
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 60s 48ms/step - accuracy: 0.6076 - lo
Epoch 4/25
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 55s 44ms/step - accuracy: 0.6499 - lo
Epoch 5/25
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 83s 44ms/step - accuracy: 0.6866 - lo
Epoch 6/25
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 80s 43ms/step - accuracy: 0.7184 - lo
Epoch 7/25
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 84s 44ms/step - accuracy: 0.7438 - lo
Epoch 8/25
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 55s 44ms/step - accuracy: 0.7688 - lo
```

```
model.history.history.keys()
```

```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

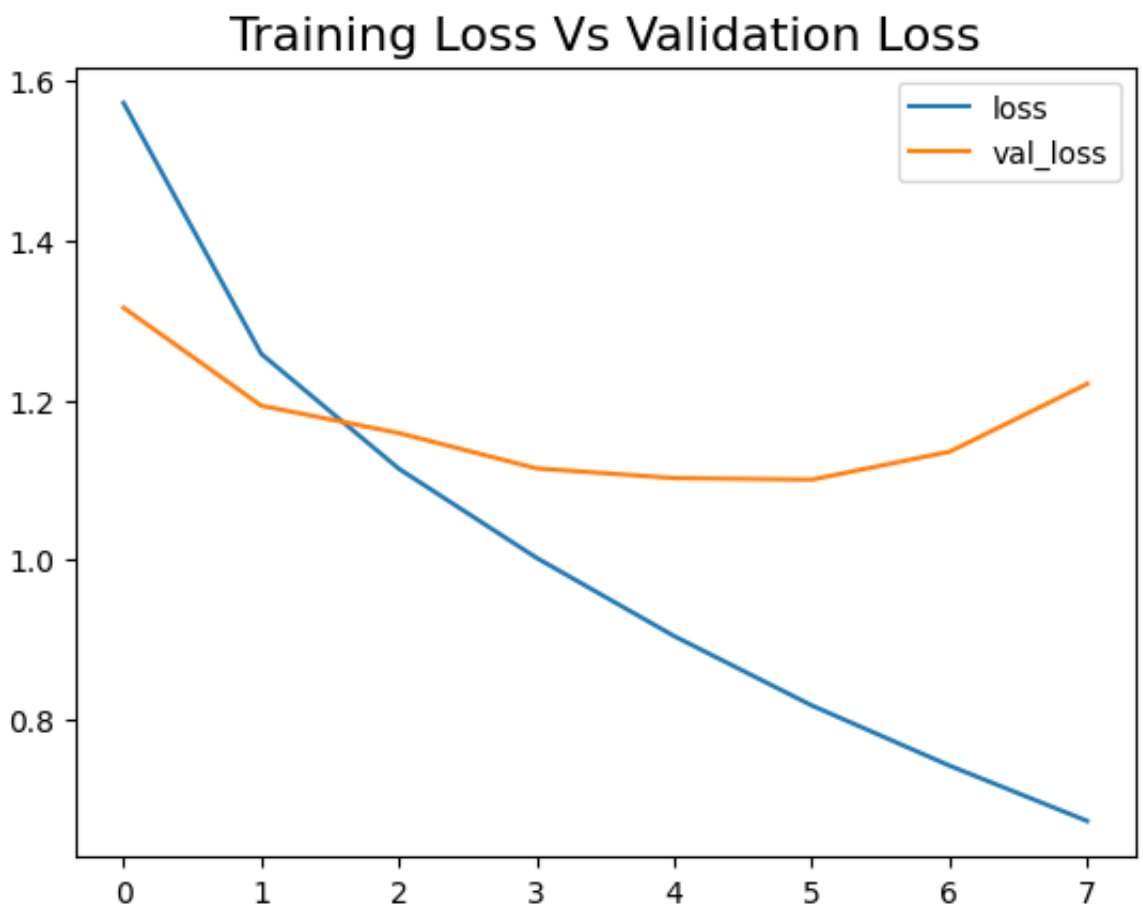
```
metrics = pd.DataFrame(model.history.history)
```

```
metrics
```

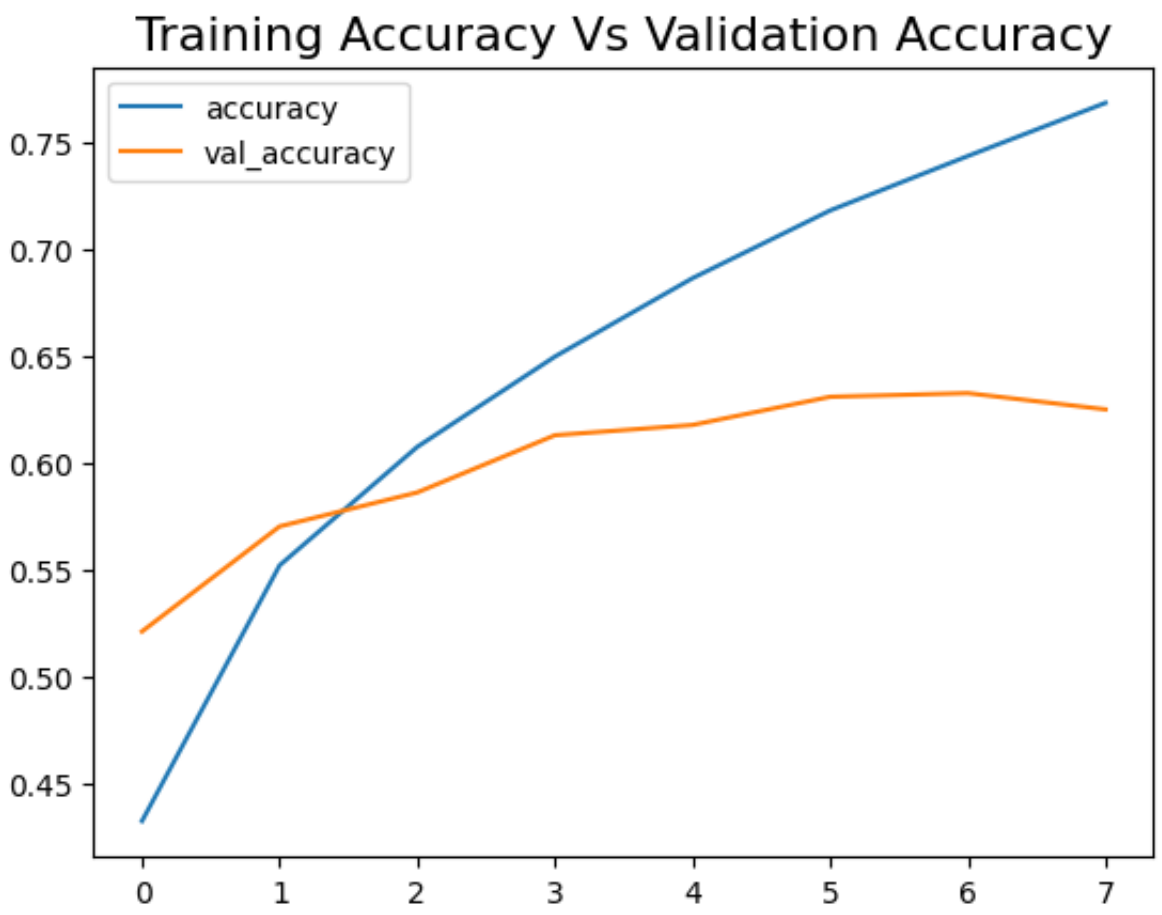
	accuracy	loss	val_accuracy	val_loss
0	0.432425	1.572079	0.5212	1.315594
1	0.552175	1.257815	0.5704	1.192931
2	0.607650	1.113969	0.5863	1.158620
3	0.649925	1.002379	0.6131	1.114410
4	0.686650	0.904382	0.6179	1.102248
5	0.718375	0.817507	0.6311	1.100356
6	0.743850	0.742191	0.6328	1.135300
7	0.768750	0.672911	0.6252	1.220290

Next steps: Generate code with metrics New interactive sheet

```
metrics[['loss', 'val_loss']].plot()
plt.title('Training Loss Vs Validation Loss', fontsize=16)
plt.show()
```



```
metrics[['accuracy', 'val_accuracy']].plot()
plt.title('Training Accuracy Vs Validation Accuracy', fontsize=16)
plt.show()
```



Validating on Test Data

```
model.evaluate(x_test,y_cat_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step - accuracy: 0.6142 - loss:
[1.2367902994155884, 0.6141999959945679]
```

Classification Report and Confusion Matrix

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
#predictions = model.predict_classes(x_test)
predictions = np.argmax(model.predict(x_test), axis=-1)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step
```

```
print(classification_report(y_test,predictions))
```

```
              precision    recall  f1-score   support

           0       0.67      0.68      0.67      1000
           1       0.76      0.72      0.74      1000
           2       0.51      0.51      0.51      1000
           3       0.35      0.65      0.45      1000
           4       0.62      0.51      0.56      1000
           5       0.60      0.41      0.49      1000
           6       0.74      0.64      0.69      1000
           7       0.78      0.62      0.69      1000
           8       0.70      0.74      0.72      1000
           9       0.72      0.66      0.69      1000

    accuracy                           0.61     10000
   macro avg       0.64      0.61      0.62     10000
weighted avg       0.64      0.61      0.62     10000
```

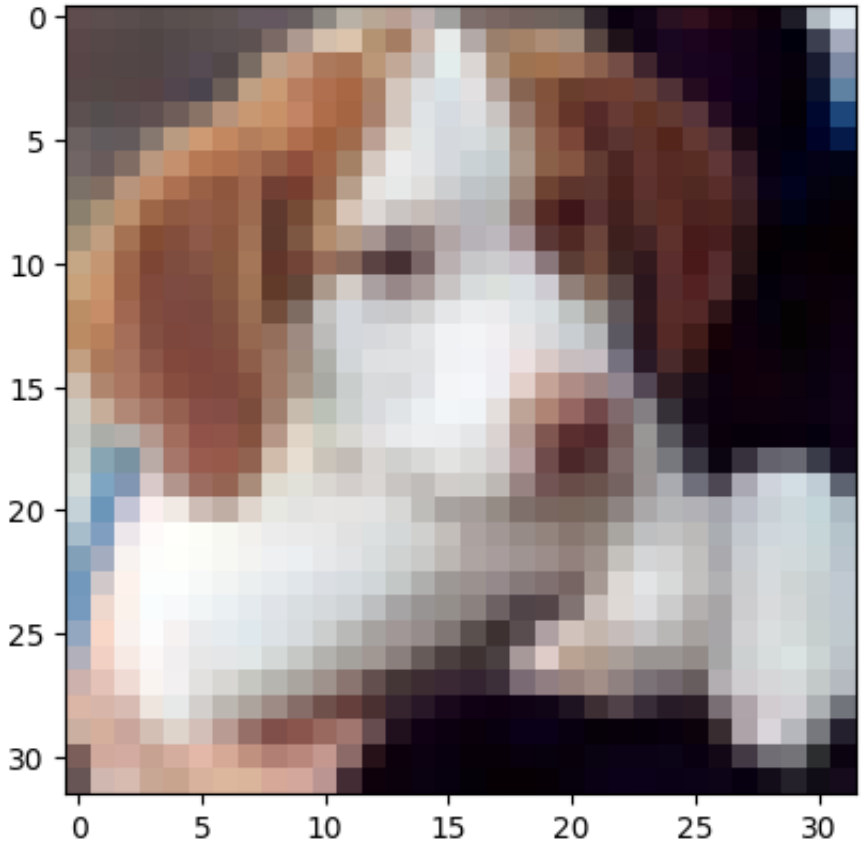
```
confusion_matrix(y_test,predictions)
```

```
array([[681,  27,  71,  47,  13,   8,   8,  10,  96,  39],
       [ 34, 718,  14,  34,   5,   8,  13,   5,  60, 109],
       [ 70,  11, 514, 184,  70,  42,  59,  16,  27,   7],
       [ 23,   9,  77, 648,  51,  83,  48,  19,  23,  19],
       [ 26,   4, 132, 158, 510,  33,  54,  55,  24,   4],
       [ 15,   5,  65, 370,  52, 413,  20,  38,  12,  10],
       [  6,  11,  77, 168,  43,  25, 644,   7,  12,   7],
       [ 16,   6,  37, 156,  71,  60,   7, 618,   7,  22],
       [110,  31,  14,  45,   5,   7,   6,   3, 741,  38],
       [ 37, 122,  13,  67,   6,  13,  10,  23,  54, 655]])
```

Predicting on single image

```
plt.imshow(x_test[16])
```

```
<matplotlib.image.AxesImage at 0x7b0db02a0350>
```




```
my_image = x_test[16]
```

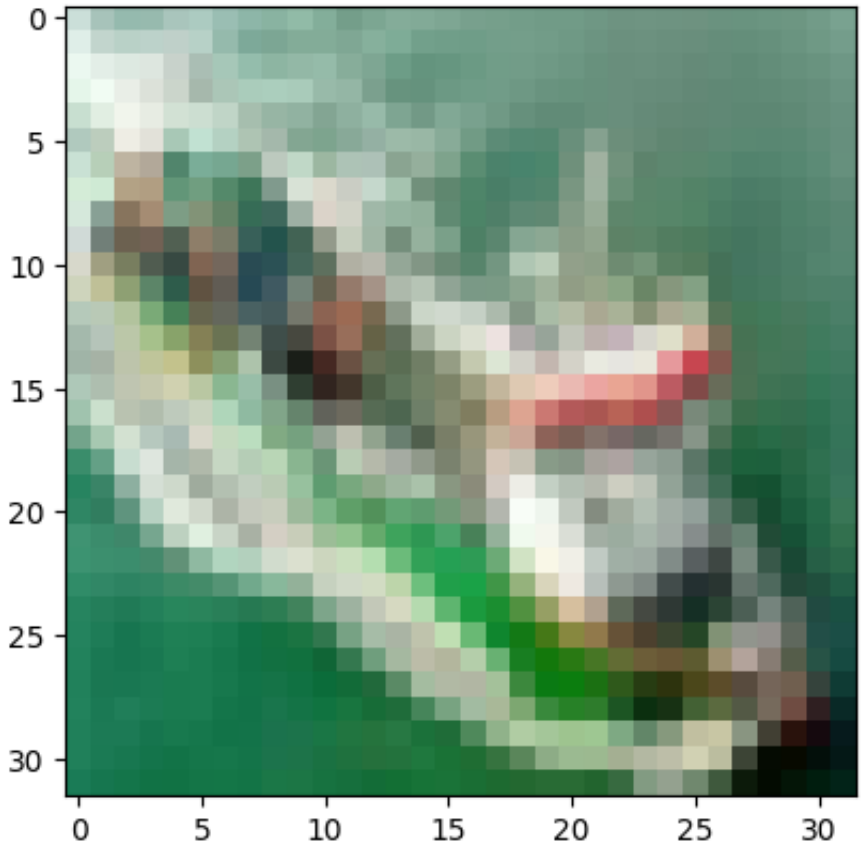
```
index = 15   # change this to any value from 1 to 15

# Show image
plt.imshow(x_test[index])
plt.show()

# Select image
my_image = x_test[index]

# Predict
pred_probs = model.predict(my_image.reshape(1, 32, 32, 3))
pred_class = np.argmax(pred_probs, axis=-1)[0]

# Print results
print("Predicted class index:", pred_class)
print("Predicted label:", LABEL_NAMES[pred_class])
print("Actual label:", LABEL_NAMES[y_test[index][0]])
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step
Predicted class index: 8
Predicted label: ship
Actual label: ship
```

```
LABEL_NAMES[y_test[16][0]]
```

```
'dog'
```

Start coding or generate with AI.